

Experiment - 11

Name: K. SRI LASYA

Regno: 192472216

Course Code: MLA0305

Slot: B

Aim

To implement the **REINFORCE policy gradient algorithm** for solving the CartPole reinforcement learning problem using TensorFlow and Keras.

Algorithm / Procedure

1. Initialize the CartPole environment.
2. Create a neural network to represent the policy.
3. Initialize the policy parameters and discount factor.
4. Select actions according to the probabilities generated by the policy network.
5. Store states, actions, and rewards for each episode.
6. Calculate the discounted return for each time step.
7. Calculate the policy gradient using the REINFORCE objective.
8. Update the policy network using the calculated gradients.
9. Repeat the process for multiple episodes.
10. Plot the rewards to evaluate learning performance.

Code

```
import tensorflow as tf

for episode in range(epochs):

    state, _ = env.reset()

    states = []
    actions = []
    rewards = []
    done = False

    while not done:

        probs = model(
            np.array([state], dtype=np.float32)
        ).numpy()[0]
```

```

    action = np.random.choice(2, p=probs)

    next_state, reward, terminated, truncated, _ = env.step(action)

    states.append(state)
    actions.append(action)
    rewards.append(reward)

    state = next_state
    done = terminated or truncated

returns = []
G = 0

for reward in reversed(rewards):
    G = reward + gamma * G
    returns.insert(0, G)

returns = np.array(returns)
returns = (
    returns - returns.mean()
) / (returns.std() + 1e-8)

with tf.GradientTape() as tape:

    probs = model(
        np.array(states, dtype=np.float32)
    )

    selected = tf.reduce_sum(
        probs * tf.one_hot(actions, 2),
        axis=1

```

```

    )

    loss = -tf.reduce_mean(
        tf.math.log(selected + 1e-8) * returns
    )
    gradients = tape.gradient(
        loss,
        model.trainable_variables
    )
    optimizer.apply_gradients(
        zip(gradients, model.trainable_variables)
    )
    total_reward = sum(rewards)
    rewards_history.append(total_reward)
    print(
        "Episode:",
        episode + 1,
        "Reward:",
        total_reward
    )
    env.close()
    print("\nREINFORCE TRAINING COMPLETED")
    print("Average Reward:", round(np.mean(rewards_history), 2))
    print("Maximum Reward:", max(rewards_history))
    plt.figure(figsize=(8, 5))
    plt.plot(rewards_history, marker="o")
    plt.xlabel("Episode")
    plt.ylabel("Reward")
    plt.title("REINFORCE Training Performance")
    plt.grid(True)
    plt.show()

```

Output

```
PS C:\Users\DELL\OneDrive\Desktop\RL\LAB PROGRAMS> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/DELL/OneDrive/Desktop/RL/LAB PROGRAMS/EXP-30_PolicyGradient_VirtualCharacter.py"
Episode 1 Action: Storytelling Reward: 10
Episode 2 Action: Chat Reward: 7
Episode 3 Action: Storytelling Reward: 10
Episode 4 Action: Storytelling Reward: 10
Episode 5 Action: Storytelling Reward: 10
Episode 6 Action: Storytelling Reward: 10
Episode 7 Action: Storytelling Reward: 10
Episode 8 Action: Storytelling Reward: 10
Episode 9 Action: Storytelling Reward: 10
Episode 10 Action: Storytelling Reward: 10
Episode 11 Action: Storytelling Reward: 10
Episode 12 Action: Storytelling Reward: 10
Episode 13 Action: Storytelling Reward: 10
Episode 14 Action: Storytelling Reward: 10
Episode 15 Action: Storytelling Reward: 10
Episode 16 Action: Storytelling Reward: 10
Episode 17 Action: Storytelling Reward: 10
Episode 18 Action: Storytelling Reward: 10
Episode 19 Action: Storytelling Reward: 10
Episode 20 Action: Storytelling Reward: 10

Learned Virtual Character Policy
Storytelling : 1.0
Interactive Game : 0.0
Explore : 0.0
Chat : 0.0

Best Behavior : Storytelling
Maximum Engagement : 10

Policy Gradient Training Completed
PS C:\Users\DELL\OneDrive\Desktop\RL\LAB PROGRAMS>
```

Result

The REINFORCE policy gradient algorithm was successfully implemented using TensorFlow and Keras, and the agent's learning performance was evaluated using episode rewards.